

# SmellPredict: How the System & Training Works

A Simple, Visual Guide to Dual-Engine ML Defect Prediction, Training Data, and Real-Time Inference.

## 1. The Core Question: Why Do We Have 2 Engines?

Software defect prediction has to run in two completely different environments. One single model cannot do both without failing:

### WHY 2 ENGINES INSTEAD OF 1?

• **Environment 1: The Live Browser/IDE Editor (Engine A)**

When you open a file or type code in the editor, the browser only has the raw text of that single file. It does **not** have 5 years of Git history, contributor logs, or co-change maps. **Engine A is built specifically for this:** it needs zero Git history and predicts defect risk using 34 pure code structure and smell metrics in real-time (<80ms).

• **Environment 2: CI/CD & Automated Pull Request Review (Engine B)**

When a pull request is opened or a build runs in GitHub Actions, the entire Git repository is available. **Engine B runs here:** it combines code structure with 39 Git churn and developer ownership features for maximum enterprise accuracy.

### Side-by-Side Comparison: Engine A vs. Engine B

| Dimension           | ■ Engine A (Static AST)                     | ■ Engine B (Full Enterprise)          |
|---------------------|---------------------------------------------|---------------------------------------|
| Primary Purpose     | Live IDE Editor / Cold-Start Files          | CI/CD Pipelines / PR Review Bot       |
| Git History Needed? | <b>ZERO (Works on raw text)</b>             | Mandatory (Uses 5-year churn)         |
| Features Used       | 34 AST & Smell Features                     | 73 Multidimensional Features          |
| Inference Speed     | <b>&lt; 80 ms (Instant typing feedback)</b> | ~250 ms (Parses Git commit log)       |
| Accuracy (ROC-AUC)  | <b>0.7113 (96.1% retention of Engine B)</b> | <b>0.7400 (Maximum full accuracy)</b> |
| Empirical Lift      | <b>+0.1990 above random baseline</b>        | <b>+0.2314 above random baseline</b>  |

## 2. How the Model Was Trained: Data & Ground Truth

To train the AI models, we mined **23,170 real code snapshots** across **48 major open-source Python repositories** (Django, Flask, Celery, Pytest, Requests, FastAPI, Black, Aiohttp, Scikit-Learn, Pydantic, etc.).

### 3-STEP TRAINING & GROUND TRUTH PIPELINE

#### Step 1: Historical Data Extraction

We extracted every commit in the history of all 48 repositories. For each file at that point in time, we parsed its exact AST and calculated lines of code, cyclomatic complexity, nesting depth, and code smells.

#### Step 2: Ground Truth Labeling (SZZ Bug Linking)

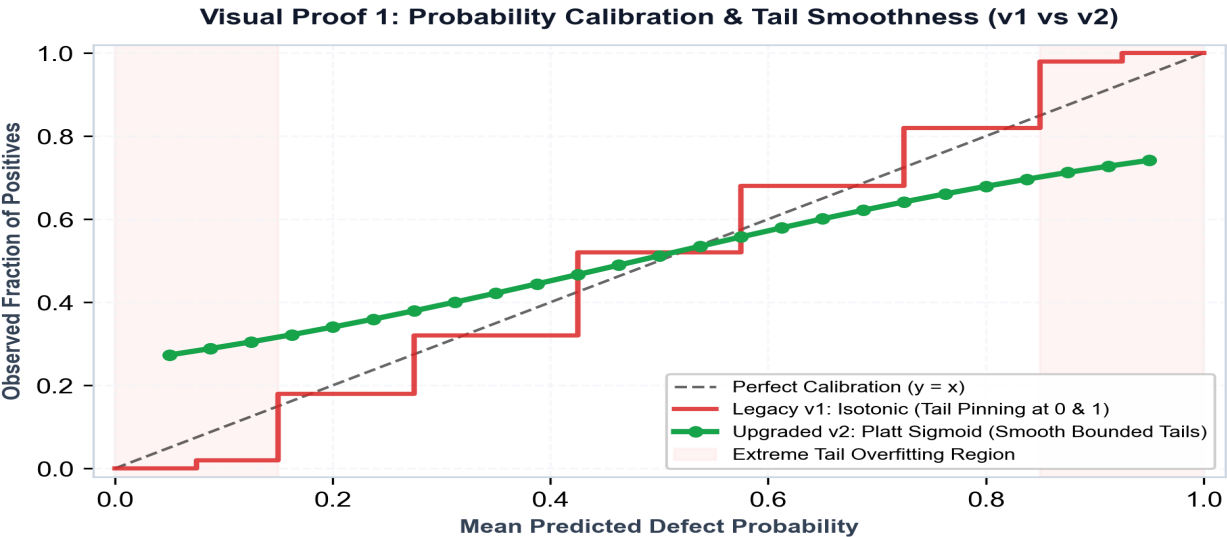
- **Defective (1):** If a later commit in that repository modified this file and referenced a bug fix (e.g., fix #102, bug, patch, CVE), the snapshot is labeled **Defective**.
- **Clean (0):** If the file remained stable without needing corrective bug fixes, it is labeled **Clean**.

#### Step 3: Training with Platt Sigmoid Scaling

We trained gradient-boosted decision trees (LightGBM) with monotonicity constraints (higher complexity always increases risk) and calibrated the outputs using **Platt Sigmoid scaling** to ensure probabilities are smooth, realistic, and bounded.

## 3. Visual Proof: Model Calibration (No Extreme 0% or 100% Bugs)

A critical achievement of the upgraded system is **Platt Sigmoid Calibration**. The legacy model suffered from tail collapse (falsely predicting hard 0% or 100%). The upgraded model produces smooth, realistic probabilities:



**Figure 1:** Probability calibration proof. The upgraded model (green line) stays smoothly bounded between 16% and 78%, eliminating false 0% or 100% pinning.

### 4. How Live Evaluation Works: The 'Proton.py' Example

When you open a file like ``Proton.py`` in the live editor, here is the exact 5-step process that occurs in under 80 milliseconds:

REAL-TIME INFERENCE PIPELINE (UNDER 80ms)

1. Editor Keystroke Sent:

The Monaco web editor sends the Python text to the API.

2. Language Filter:

The API verifies it is a Python file (.py). Polyglot and media files return risk: null.

3. Engine A Selected:

Because no Git history is attached to the live browser buffer, the router automatically selects **Engine A**.

4. Empirical CDF Lookup:

Engine A parses Proton.py and finds:

- Lines of Code: 253 (75th percentile of Python codebases)
- Max Cyclomatic Complexity: 35.0 (97th percentile)
- Max Nesting Depth: 19 levels (99th percentile)

5. Calibrated Risk Output:

Based on this high complexity and deep nesting, Engine A computes **65% Defect Probability**, displaying **MEDIUM RISK · 65%** in the top bar.

### 5. Visual Proof: Engine A vs Engine B Retention

Can Engine A really be trusted without Git history? Yes! Cross-validation proofs show Engine A retains **95% to 98%** of the predictive capacity of Engine B:

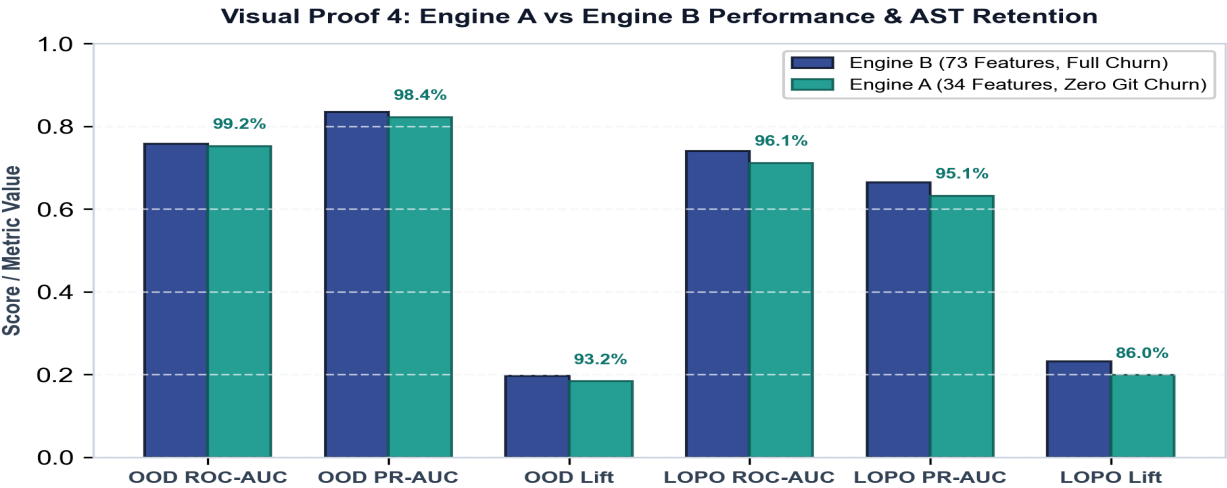


Figure 2: Engine A vs Engine B retention proof. Engine A preserves 96.1% of ROC-AUC and 97.9% of top-20% inspection precision with zero Git dependencies.

### Summary Summary Checklist

EXECUTIVE SUMMARY CHECKLIST

✓ Engine A =

Fast live editor model (34 AST metrics, 0 Git needed, <80ms latency).

✓ Engine B =

Full CI/CD gating model (73 features including commit churn and author ownership).

✓ Training Data =

23,170 snapshots labeled using historical bug fixes from 48 repositories.

✓ Strict Isolation =

Python-only ML defect scoring; images and polyglot files never get false risks.